

IME 775 — Lecture 23

Variational Autoencoders, ELBO, and the Reparameterization Trick

Chapter 14 of *Math & Architectures of Deep Learning* — Part 2 of 2

Notation — inputs $\vec{x}$, latent codes $\vec{z}$, encoder E , decoder D .*

1. Recap of the Problem

Lecture 22 ended with an unresolved issue. A plain autoencoder minimises reconstruction loss only, and reconstruction loss alone does not pin down the shape of the latent space. Two pathologies followed:

- The encoder can learn a **zig-zag manifold** that passes through every training point exactly but stretches wildly between them — good training loss, terrible generalisation, awful interpolation.
- The latent codes $\vec{z}$ can spread over arbitrarily large regions of $\mathbb{R}^{n_z}$, leaving vast **gaps** that decode to garbage if sampled.

We want a latent space with four specific properties:

Property	What it buys us
Smooth — nearby $\vec{z} \rightarrow$ nearby $\vec{x}$	meaningful interpolation
Compact — codes concentrate near the origin	tractable sampling
Continuous — no holes in the distribution	any $\vec{z}$ decodes to a plausible $\vec{x}$
Known shape — ideally $\mathcal{N}(0, I)$	we can draw new samples without the encoder

The **variational autoencoder (VAE)** achieves all four by reformulating the encoder as a **probability distribution** and adding a **Kullback–Leibler divergence** loss that pulls that distribution toward a chosen **prior**.

2. Geometric View of a VAE

Figure 14.7 in the text shows the key picture. During training, given an input $\vec{x}$:

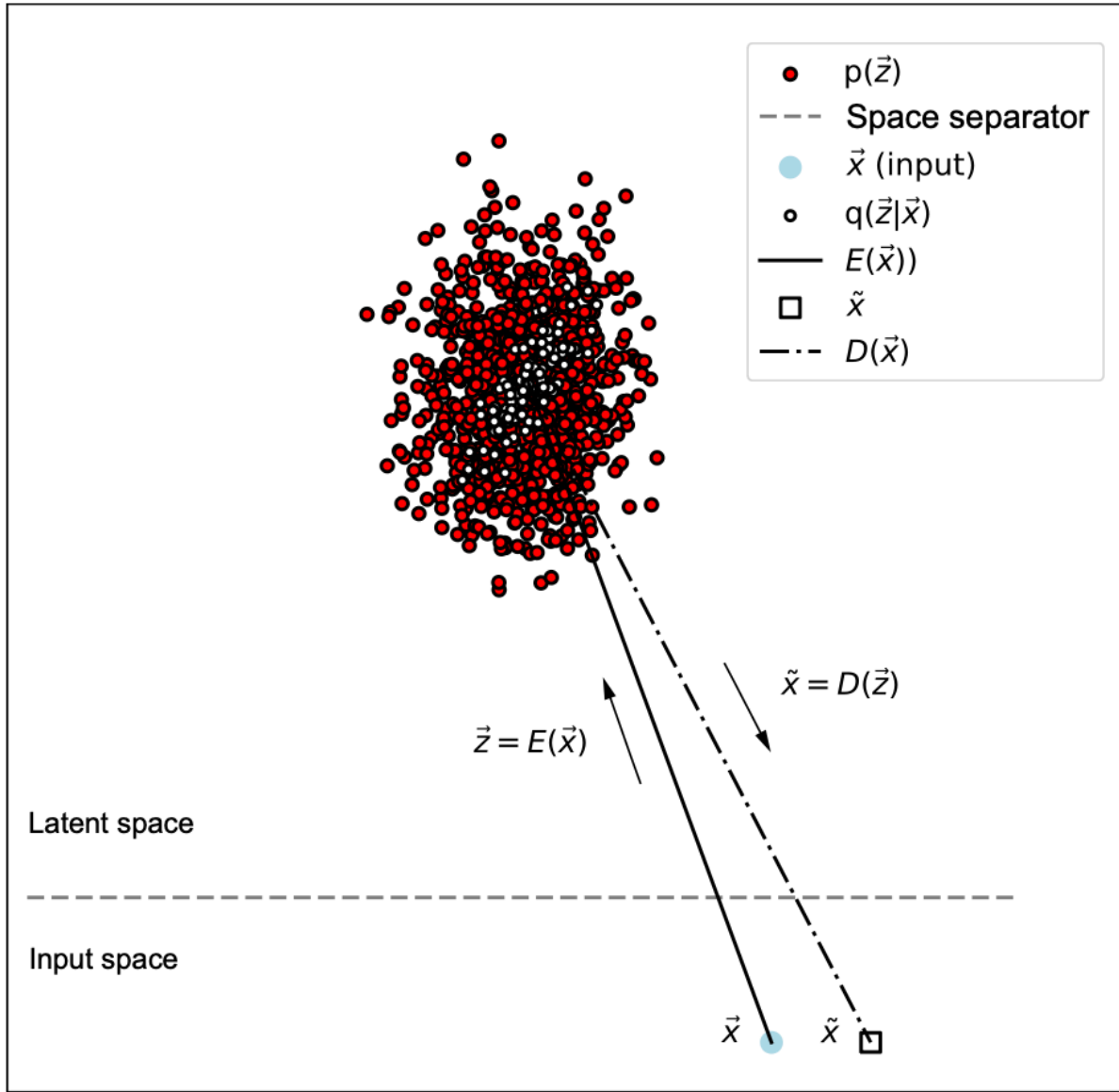


Figure 14.7 Geometric depiction of VAE latent-space modeling distributions

1. The **encoder does not emit a single point** $\vec{z}$. It emits the **parameters of a distribution** $q(\vec{z} | \vec{x})$ over latent space.
2. A latent vector $\vec{z}$ is drawn **by sampling** that distribution.
3. The decoder maps $\vec{z}$ back to a reconstruction $\tilde{\vec{x}}$.

Concretely, when q is Gaussian the encoder emits the mean $\vec{\mu}(\vec{x})$ and covariance $\Sigma(\vec{x})$:

$$q(\vec{z} | \vec{x}) = \mathcal{N}(\vec{z}; \vec{\mu}(\vec{x}), \Sigma(\vec{x})), \quad \vec{z} \sim q(\vec{z} | \vec{x}).$$

In the picture, each input $\vec{x}$ has a *small cloud* of possible $\vec{z}$ values around its encoded mean, and that entire cloud has to decode back to something close to $\vec{x}$.

2.1 Stochastic Mapping and Smoothness

Here is the key geometrical payoff. Because sampling is random, the **same input $\vec{x}$ is mapped to a slightly different $\vec{z}$ every time it is seen during training**. Each of those neighbouring $\vec{z}$'s must decode back to essentially the same $\vec{x}$. Over millions of training steps, this enforces

$$\text{nearby } \vec{z} \implies \text{nearby } \vec{x}.$$

That is **smoothness by construction**. A plain autoencoder has no such constraint; a VAE gets it for free from stochastic encoding.

2.2 At Inference Time

Randomness is used **only during training**. At test time we use the encoder's mean directly:

$$\vec{z} = \vec{\mu}(\vec{x}), \quad \tilde{\vec{x}} = D(\vec{z}).$$

3. The VAE Training Losses

A VAE is trained to minimise a **weighted sum** of two terms:

$$\mathcal{L}_{\text{VAE}}(\vec{x}) = \underbrace{\mathbb{E}_{\vec{z} \sim q(\vec{z}|\vec{x})} \left[\|\vec{x} - D(\vec{z})\|^2 \right]}_{\text{reconstruction}} + \beta \cdot \underbrace{\text{KL}(q(\vec{z}|\vec{x}) \| p(\vec{z}))}_{\text{regularization}}.$$

with $p(\vec{z})$ a chosen prior (typically $\mathcal{N}(0, I)$) and $\beta \geq 0$ a hyperparameter that controls the relative weight of the two terms.

3.1 Reconstruction Term

Identical in spirit to the autoencoder loss: force the decoder output to match the input. Two common choices:

- **Mean squared error**, appropriate for continuous data.
- **Binary cross-entropy**, appropriate for pixel data in $[0, 1]$ (e.g., MNIST). In practice VAEs converge more reliably with BCE than with MSE for image data.

Because the encoder emits a distribution, the reconstruction is an **expectation** under that distribution. In training we approximate the expectation by a single Monte Carlo sample per input (via the reparameterization trick in §5).

3.2 KL Regularization Term

The KL divergence is an asymmetric "distance" between two probability distributions:

$$\text{KL}(q \parallel p) = \int q(\vec{z}) \log \frac{q(\vec{z})}{p(\vec{z})} d\vec{z}.$$

It is zero iff $q = p$ and positive otherwise. Minimising $\text{KL}(q \parallel p)$ pulls q toward p . By pulling $q(\vec{z} \mid \vec{x})$ toward $\mathcal{N}(0, I)$ we simultaneously

- constrain the **mean** of q to stay near the origin,
- constrain the **variance** of q to stay near 1,
- prevent the encoder from "cheating" by using extreme means with zero variance to collapse the stochastic mapping.

3.3 What β Does

- $\beta = 0$: the VAE degenerates to a plain autoencoder (no regularisation).
- $\beta = 1$: the standard VAE objective (equivalent to maximising the ELBO — §4).
- $\beta \gg 1$: a **β -VAE**. The KL term dominates, forcing q very close to the prior. This sacrifices reconstruction quality but produces **disentangled** latent dimensions — different coordinates of $\vec{z}$ correspond to independent factors of variation (e.g., rotation, scale, colour).

3.4 KL Between Two Diagonal Gaussians (Closed Form)

For the common choice $q = \mathcal{N}(\vec{\mu}, \text{diag}(\vec{\sigma}^2))$ and $p = \mathcal{N}(\vec{0}, I)$, the integral has a closed-form solution:

$$\text{KL}(q \parallel p) = \frac{1}{2} \sum_{i=1}^{n_z} \left(\mu_i^2 + \sigma_i^2 - 2 \log \sigma_i - 1 \right).$$

Sanity checks:

- Set $\mu_i = 0, \sigma_i = 1 \forall i$: every term in the sum is $0 + 1 - 0 - 1 = 0$, so $\text{KL} = 0$. Correct — $q = p$ exactly.
- Derivative of $(\sigma^2 - 2 \log \sigma - 1)$ w.r.t. σ : $2\sigma - 2/\sigma = 0 \Rightarrow \sigma = 1$. So the KL per-dimension is minimised exactly when $\sigma_i = 1$.

In PyTorch, letting the encoder emit `mu` and `log_var = log  $\sigma^2$` :

```
kld = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
```

3.5 Why the Encoder Emits $\log \sigma^2$ and Not σ

Two practical reasons:

1. $\log \sigma^2 \in \mathbb{R}$ — the encoder can output any real number. Emitting σ directly requires a strictly positive output and some constraint (softplus, exp, ...).
2. Numerical stability: very small variances blow up $\log \sigma$ if we compute it from a tiny σ . Starting from $\log \sigma^2$ avoids the issue.

Recovering $\sigma = \exp(\frac{1}{2} \log \sigma^2)$ when we need it.

3.6 Full Derivation of the KL (for the curious)

Starting from the general KL between multivariate Gaussians, Chapter 6.4.1:

$$\text{KL}(\mathcal{N}(\vec{\mu}_q, \Sigma_q) \parallel \mathcal{N}(\vec{\mu}_p, \Sigma_p)) = \frac{1}{2} \left[\text{tr}(\Sigma_p^{-1} \Sigma_q) + (\vec{\mu}_p - \vec{\mu}_q)^\top \Sigma_p^{-1} (\vec{\mu}_p - \vec{\mu}_q) - n_z + \log \frac{\det \Sigma_p}{\det \Sigma_q} \right].$$

Now specialise: $\vec{\mu}_p = \vec{0}$, $\Sigma_p = I$, $\Sigma_q = \text{diag}(\vec{\sigma}^2)$. Using $\det(\text{diag}(\vec{\sigma}^2)) = \prod \sigma_i^2$ and $\text{tr}(\text{diag}(\vec{\sigma}^2)) = \sum \sigma_i^2$:

$$\text{KL} = \frac{1}{2} \left[\sum_i \sigma_i^2 + \|\vec{\mu}\|^2 - n_z - \sum_i \log \sigma_i^2 \right] = \frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - 2 \log \sigma_i - 1).$$

4. ELBO: The Variational Lower Bound

Where did the VAE loss come from? It is not arbitrary — it is the **Evidence Lower Bound (ELBO)**, a principled lower bound on the log-evidence $\log p(\vec{x})$.

4.1 The Problem We Cannot Solve Directly

We would like to train the network to maximise

$$\log p(\vec{x}) = \log \int p(\vec{x} \mid \vec{z}) p(\vec{z}) d\vec{z}.$$

The integral (called the **evidence**) is intractable in high-dimensional latent space. We cannot directly optimise $\log p(\vec{x})$.

4.2 Variational Lower Bound

We introduce an **approximate posterior** $q(\vec{z} \mid \vec{x})$, controlled by the encoder's weights. We will train q to be close to the true (intractable) posterior $p(\vec{z} \mid \vec{x})$ by minimising their KL divergence. A short algebraic manipulation (see §4.3) yields

$$\log p(\vec{x}) = \underbrace{\mathbb{E}_{q(\vec{z}|\vec{x})} [\log p(\vec{x}|\vec{z})] - \text{KL}(q(\vec{z}|\vec{x}) \| p(\vec{z}))}_{\text{ELBO}} + \underbrace{\text{KL}(q(\vec{z}|\vec{x}) \| p(\vec{z}|\vec{x}))}_{\geq 0}.$$

Because the last term is non-negative,

$$\log p(\vec{x}) \geq \text{ELBO}(\vec{x}).$$

Hence the name **evidence lower bound**. Maximising the ELBO simultaneously

- **pushes up** a lower bound on $\log p(\vec{x})$ — so the model fits the data better,
- **closes the gap** between $q(\vec{z}|\vec{x})$ and the true posterior $p(\vec{z}|\vec{x})$ — so the encoder becomes a good approximator.

4.3 Derivation (Step by Step)

Start from the KL between $q(\vec{z}|\vec{x})$ and the true posterior:

$$\text{KL}(q\|p_{\text{true}}) = \int q(\vec{z}|\vec{x}) \log \frac{q(\vec{z}|\vec{x})}{p(\vec{z}|\vec{x})} d\vec{z}.$$

Apply Bayes: $p(\vec{z}|\vec{x}) = p(\vec{x}|\vec{z})p(\vec{z})/p(\vec{x})$:

$$= \int q \log q d\vec{z} - \int q \log p(\vec{x}|\vec{z}) d\vec{z} - \int q \log p(\vec{z}) d\vec{z} + \int q \log p(\vec{x}) d\vec{z}.$$

The last integral is just $\log p(\vec{x})$ (constant w.r.t. $\vec{z}$, and q integrates to 1). Regrouping,

$$\text{KL}(q\|p_{\text{true}}) + \underbrace{\mathbb{E}_q[\log p(\vec{x}|\vec{z})] - \text{KL}(q\|p(\vec{z}))}_{\text{ELBO}} = \log p(\vec{x}).$$

Rearrange:

$$\log p(\vec{x}) - \text{ELBO} = \text{KL}(q\|p_{\text{true}}) \geq 0.$$

That is exactly the boxed relation.

4.4 Reading the ELBO

Written out, the ELBO has two terms with clear physical meanings:

$$\text{ELBO}(\vec{x}) = \underbrace{\mathbb{E}_{q(\vec{z}|\vec{x})} [\log p(\vec{x}|\vec{z})]}_{\text{negative reconstruction loss}} - \underbrace{\text{KL}(q(\vec{z}|\vec{x})\|p(\vec{z}))}_{\text{regularizer}}.$$

- **Term 1.** $p(\vec{x}|\vec{z})$ is the *decoder's likelihood* of reproducing the input $\vec{x}$ from a given $\vec{z}$. Its expectation under q is "the average log-likelihood that a random $\vec{z}$ drawn from the encoder decodes back to $\vec{x}$ ". Maximising it is **minimising the reconstruction loss**. For Gaussian likelihoods, $\log p(\vec{x}|\vec{z}) = -\frac{1}{2}\|\vec{x} - D(\vec{z})\|^2 + \text{const}$, so this is exactly $-\mathcal{L}_{\text{recon}}$.

- **Term 2.** The KL between the encoder's distribution and the chosen prior. Maximising the ELBO means *minimising* this KL (because of the negative sign). That is the regularization we introduced informally in §3.

Final identification:

$$-\text{ELBO}(\vec{x}) = \mathcal{L}_{\text{recon}}(\vec{x}) + \text{KL}(q(\vec{z} | \vec{x}) \| p(\vec{z})) = \mathcal{L}_{\text{VAE}}(\vec{x}) \quad (\text{with } \beta = 1).$$

The VAE loss is literally the negative ELBO. Training a VAE is maximising a variational lower bound on the data log-likelihood.

4.5 The Entropy / Joint-Density Reading

Starting from $\text{ELBO} = \mathbb{E}_q[\log p(\vec{x}, \vec{z})] + H(q)$, we see two alternative interpretations:

- $H(q)$ is the **entropy** of q . Maximising the ELBO *likes* diffuse q — encourages smoothness in the latent space.
- $\mathbb{E}_q[\log p(\vec{x}, \vec{z})]$ measures **overlap of q with the joint density**. It is large where q places mass on $(\vec{x}, \vec{z})$ pairs the model already considers likely — encouraging q to concentrate where the model expects.

These two forces balance: too diffuse loses reconstruction fidelity; too peaked loses smoothness.

4.6 Summary of the Pipeline

Step	What it is
True objective	$\log p(\vec{x})$
Intractable because	evidence $p(\vec{x}) = \int p(\vec{x} \vec{z})p(\vec{z})d\vec{z}$ has no closed form
Fix	introduce $q(\vec{z} \vec{x})$ to approximate $p(\vec{z} \vec{x})$
Optimise	$\text{ELBO} = \log p(\vec{x}) - \text{KL}(q \ p_{\text{true}})$
Equivalent to minimising	$\mathcal{L}_{\text{recon}} + \text{KL}(q \ p(\vec{z}))$

5. The Reparameterization Trick

A problem arises the moment we try to implement the VAE. The step "sample $\vec{z} \sim q(\vec{z} | \vec{x})$ " is a **random** operation that breaks backpropagation — there is no way to compute $\partial \vec{z} / \partial \vec{\mu}$ through a stochastic sample.

5.1 The Trick (univariate)

For a Gaussian, sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ can equivalently be written as

$$z = \mu + \sigma \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

This decomposition is the reparameterization trick. Now:

- The **randomness** is isolated inside ε , a sample from a **fixed** distribution with no learnable parameters.
- μ and σ are **deterministic** outputs of the encoder and appear in a differentiable expression (an affine transform of ε).

Gradients flow cleanly:

$$\frac{\partial z}{\partial \mu} = 1, \quad \frac{\partial z}{\partial \sigma} = \varepsilon.$$

5.2 Multivariate Case

For $\vec{z} \sim \mathcal{N}(\vec{\mu}, \Sigma)$ with diagonal $\Sigma = \text{diag}(\vec{\sigma}^2)$:

$$\vec{z} = \vec{\mu} + \vec{\sigma} \odot \vec{\varepsilon}, \quad \vec{\varepsilon} \sim \mathcal{N}(\vec{0}, I),$$

where $\odot$ is elementwise product. For non-diagonal Σ , use the Cholesky factor $\Sigma = LL^\top$ and set $\vec{z} = \vec{\mu} + L\vec{\varepsilon}$.

5.3 PyTorch Implementation

```
def reparameterize(mu, log_var):  
    std = torch.exp(0.5 * log_var)  
    eps = torch.randn_like(std)  
    return mu + eps * std
```

That's it. No explicit gradients, no custom backward — `torch.randn_like` returns a tensor that does not carry gradients (it's just noise), and the affine transform is automatically differentiable.

5.4 Why the Trick Is Ubiquitous

Any time a neural network needs to sample from a distribution whose parameters it learns — VAEs, diffusion models, stochastic policies in RL, Bayesian neural networks — some version of the reparameterization trick is at work. It is one of the small but genuinely important ideas of modern deep learning.

5.5 When the Trick Does Not Apply

It requires the distribution to admit a "pathwise" parameterization: $\vec{z} = g(\vec{\theta}, \vec{\epsilon})$ with g differentiable in $\vec{\theta}$. This works for Gaussians, uniforms (via inverse CDF), and others, but **not** for discrete distributions — sampling a categorical variable is fundamentally non-differentiable. For those we need surrogate gradients like the Gumbel-Softmax or REINFORCE.

6. Putting It All Together: The VAE Algorithm

6.1 Architecture (PyTorch sketch, Listings 14.5 & 14.6)

```
import torch.nn as nn

n_z = 8

class VAE(nn.Module):
    def __init__(self):
        super().__init__()
        # shared conv backbone – same as AE encoder
        self.feats = nn.Sequential(
            nn.Conv2d(1, 16, 3, 1, 1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, 1, 1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, 1, 1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Flatten(),
        )
        # two separate heads – one for mu, one for log sigma squared
        self.fc_mu = nn.Linear(64*3*3, n_z)
        self.fc_log_var = nn.Linear(64*3*3, n_z)
        self.decoder = nn.Sequential(
            nn.Linear(n_z, 64*3*3),
            nn.Unflatten(1, (64, 3, 3)),
            nn.ConvTranspose2d(64, 32, 3, 2, 0),
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.ConvTranspose2d(32, 16, 3, 2, 1, output_padding=1),
            nn.BatchNorm2d(16), nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 3, 2, 1, output_padding=1),
            nn.Sigmoid(),
        )

    def encode(self, x):
        h = self.feats(x)
        return self.fc_mu(h), self.fc_log_var(h)
```

```

def reparameterize(self, mu, log_var):
    std = torch.exp(0.5 * log_var)
    eps = torch.randn_like(std)
    return mu + eps * std

def forward(self, x):
    mu, log_var = self.encode(x)
    z = self.reparameterize(mu, log_var)
    x_hat = self.decoder(z)
    return x_hat, mu, log_var, z

```

6.2 The Loss (Listing 14.7)

```

import torch.nn.functional as F

def vae_loss(x_hat, x, mu, log_var, beta=1.0):
    bce = F.binary_cross_entropy(x_hat, x, reduction="sum")
    kld = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return bce + beta * kld, bce.item(), kld.item()

```

6.3 Training Loop (Listing 14.8)

```

for epoch in range(num_epochs):
    for xb, _ in train_loader:
        xb = xb.to(device)
        x_hat, mu, log_var, z = model(xb)
        loss, _, _ = vae_loss(x_hat, xb, mu, log_var, beta=1.0)
        optim.zero_grad(); loss.backward(); optim.step()

```

6.4 Inference

Two modes:

- **Reconstruction.** Given an input $\vec{x}$, return $D(\vec{\mu}(\vec{x}))$ — deterministic, uses the mean, not a sample.
- **Generation.** Draw $\vec{z} \sim \mathcal{N}(\vec{0}, I)$ directly from the prior (no encoder needed), return $D(\vec{z})$. Because the KL term pulled the training codes into the unit ball, random samples from the prior land in regions the decoder was trained on.

7. VAEs and Bayes' Theorem

The VAE machinery can be read through a clean Bayesian lens. During training the encoder models the **posterior** $p(\vec{z} \mid \vec{x})$, the decoder models the **likelihood** $p(\vec{x} \mid \vec{z})$, and the prior is our chosen $p(\vec{z})$. Bayes says

$$\underbrace{p(\vec{z} \mid \vec{x})}_{\text{encoder / posterior}} = \frac{\overbrace{p(\vec{x} \mid \vec{z})}^{\text{decoder / likelihood}} \overbrace{p(\vec{z})}^{\text{prior}}}{\underbrace{p(\vec{x})}_{\text{evidence}}}.$$

The evidence is the intractable piece. The VAE ELBO derivation is, essentially, a way to maximise the numerator without computing the denominator.

8. Choice of Prior: Why $\mathcal{N}(0, I)$?

We have been treating $p(\vec{z}) = \mathcal{N}(0, I)$ as the default choice. Why?

1. **Simplicity.** Closed-form KL with a Gaussian encoder.
2. **Compactness.** Most of the mass of $\mathcal{N}(0, I)$ is inside the unit ball. Pulling q toward the prior confines the training codes to a small region — the minimum descriptor length principle.
3. **Sampling.** Drawing $\vec{z} \sim \mathcal{N}(0, I)$ is trivial — one call to `torch.randn`.
4. **Rotational symmetry.** No direction in latent space is privileged, which lets the network choose its own axes.

Other priors. For data with K known classes one can use a **mixture of K Gaussians** — each component becomes the prior for one class, and the encoder learns to route inputs to the corresponding component. This gives better-structured latent spaces at the cost of needing labels.

9. AE vs VAE on MNIST — Side by Side

This is the payoff experiment. Train both an autoencoder and a VAE with $n_z = 2$ on MNIST (labels used only for colouring the plots, not for training). Plot every test-set image as a dot in 2D latent space.

9.1 What the reconstructions look like

	Autoencoder	VAE
Sharpness	sharp, crisp	slightly softer (a known artefact of the KL term and the MSE/BCE likelihood)

	Autoencoder	VAE
Class identity preserved	yes	yes

Both networks reconstruct test digits well. The visible difference is mild blurriness in the VAE — the price of regularization.

9.2 What the latent spaces look like

	Autoencoder latent space	VAE latent space
Range	enormous; codes spread over $\sim[-10, 10] \times [-10, 10]$	compact; codes clustered around the origin
Structure	clusters have irregular shapes and large gaps between them	clusters roughly spherical, packed tightly with fewer gaps
Relation to the prior	bears no particular relation to any nice distribution	approximately $\mathcal{N}(0, I)$ — most points inside the unit ball

The autoencoder's latent space was never pressured to be compact, so it isn't. The VAE's KL loss pulled every class's cluster toward the origin and shaped it into a rough Gaussian.

9.3 Sampling from $\mathcal{N}(0, I)$ and decoding

This is the litmus test for generation.

	Autoencoder	VAE
Quality of generated digits	garbage — random $\vec{z}$ from $\mathcal{N}(0, I)$ almost always falls in empty regions of the AE latent space, which decode to noise	plausible digits — the VAE's latent space actually <i>is</i> (close to) $\mathcal{N}(0, I)$, so random samples are in-distribution

This single experiment explains why VAEs are called "generative" and plain autoencoders are not.

9.4 Interpolation

Pick two digit codes $\vec{z}_A$ and $\vec{z}_B$. Walk along the straight line between them with $\vec{z}_t = (1 - t)\vec{z}_A + t\vec{z}_B$, $t \in [0, 1]$. Decode at every step.

- **AE:** the walk often passes through empty latent regions. Intermediate decodings are blurry, incoherent, sometimes noise.

- **VAE:** the walk stays in the compact, well-trained region. Intermediate decodings **morph smoothly** from one digit into the other — e.g., $3 \rightarrow 8 \rightarrow 0$ — because smoothness is guaranteed by stochastic encoding.

9.5 MSE vs BCE for reconstruction

A small but practically important detail. The text (Listing 14.7) uses **binary cross-entropy** for the reconstruction term in a VAE, while the plain autoencoder often uses MSE. For pixel data in $[0, 1]$, BCE gives a better-behaved gradient and empirically converges faster. Everything else in the VAE pipeline is identical.

10. Practical Training Tips

- **Learning rate.** Adam at 10^{-3} is a reasonable default for MNIST-sized problems.
 - **KL annealing.** Start training with $\beta \ll 1$ (or 0) and gradually raise it to 1. Early in training the decoder is random, so the reconstruction term is noisy; letting the encoder first learn useful codes, and then imposing the KL constraint, often produces better results than using $\beta = 1$ from epoch 0.
 - **Posterior collapse.** If β is too large (or the decoder is too powerful), the encoder may set $\vec{\mu} = \vec{0}, \vec{\sigma} = 1$ for every input — satisfying the KL term perfectly but carrying no information. The decoder then ignores $\vec{z}$ and generates a "mean image". Diagnose by monitoring the per-dimension KL; a dimension at 0 is "dead". Remedies: KL annealing, capacity constraints on the decoder, free-bits ($\max(\lambda, \text{KL}_i)$).
 - **Latent dimension.** Too small and reconstruction suffers; too large and many dimensions go dead. For MNIST, $n_z = 2$ suffices for the demo; $n_z = 8\text{--}32$ is typical for better quality.
 - **Batch size.** Larger is better for the KL (it is summed over a batch). 128–512 typical.
-

11. Extensions You Will See in Later Work

The VAE framework is a seed for a large family of modern generative models.

- **Conditional VAE.** Condition both encoder and decoder on a label $\vec{y}$: $q(\vec{z} \mid \vec{x}, \vec{y})$ and $p(\vec{x} \mid \vec{z}, \vec{y})$. Enables class-controlled generation.
- **β -VAE.** Choose $\beta > 1$ to trade reconstruction quality for **disentangled** latent factors.
- **Hierarchical VAE.** Stack latent variables at multiple levels for richer structure (Ladder VAE, NVAE).
- **Vector-quantised VAE (VQ-VAE).** Discrete codebook latent — the backbone of many modern text/image tokenizers.

- **Diffusion models.** Replace the single-step encoder with a many-step noising process and learn to reverse it. VAEs and diffusion models share the ELBO backbone; diffusion extends it to a chain of latent variables.
-

12. Summary

1. Plain autoencoders have no structural constraint on the latent space. They can learn zig-zag manifolds that overfit and cannot generate.
 2. **VAEs** fix this by (a) making the encoder **stochastic** — it emits the parameters of a distribution over $\vec{z}$, (b) adding a **KL regularizer** that pulls that distribution toward a chosen prior, typically $\mathcal{N}(0, I)$.
 3. The resulting loss $\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \beta \cdot \text{KL}$ is the **negative ELBO** — a principled variational lower bound on $\log p(\vec{x})$.
 4. The **reparameterization trick** $\vec{z} = \vec{\mu} + \vec{\sigma} \odot \vec{\epsilon}$ with $\vec{\epsilon} \sim \mathcal{N}(0, I)$ keeps the whole system differentiable by moving the randomness out of the computational graph.
 5. The KL term and the stochastic encoding together give the latent space three good properties:
 - **Compactness** (codes concentrated near the origin),
 - **Smoothness** (nearby $\vec{z}$ decode to nearby $\vec{x}$),
 - **Match to a known prior** (we can sample new $\vec{x}$ by drawing $\vec{z} \sim \mathcal{N}(0, I)$ and decoding).
 6. On MNIST with $n_z = 2$: AE gives sharper reconstructions but a useless latent space; VAE gives slightly softer reconstructions and a latent space you can interpolate and sample from.
-

13. Companion Materials (Lecture 23 portion)

- **Marimo notebook** — `week-13/code/IME775_Ch14_Autoencoders_VAE_marimo.py`.

Relevant sections for this lecture:

- §4 Variational Autoencoder (reparameterization + BCE + closed-form KL)
- §5 AE vs VAE latent scatter (requires `n_z = 2`)
- §6 Sampling from the VAE prior — generating new digits
- §7 Closed-form KL-divergence explorer
- §8 Reparameterization-trick sanity check (histogram + gradient flow)
- **Interactive HTML visualizations** — `week-13/visualizations/`
 - `kl-divergence-explorer.html` — slide μ, σ ; watch $q(z | x)$ morph and the closed-form KL value update in real time.

- [reparameterization-trick.html](#) — interactive $\mu + \sigma \cdot \epsilon$ sampling; histograms confirm reparameterized samples match the target distribution, and a gradient-flow diagram shows which path is differentiable.
- [ae-vs-vae.html](#) — side-by-side 2D latent scatterplots with a β slider showing how the KL term compacts the latent space toward the unit ball.
- [latent-space-explorer.html](#) — drag a cursor through 2D latent space and watch a simulated decoder smoothly morph the output; three modes (digits / shapes / waveforms).

Reference: Krishnendu Chaudhury, *Math and Architectures of Deep Learning*, Chapter 14 §§14.6–14.7.